PROGRAMACIÓN III: TRABAJO PRACTICO OBLIGATORIO



Universidad: Universidad Argentina de la Empresa

Carrera: Licenciatura en Gestión de Tecnología de la Información

Materia: Programación III

Turno: Viernes Tarde

Nombre y apellido de la profesora:

- Leon, Maria Angela

Nombre, apellido y Legajo de los integrantes del equipo:

- Francisco Ferrero - 1208322
- Josue Palacios - 1207683

Fecha de la primera entrega: 23/10/2026

Fecha de la segunda entrega: 06/11/2026

Informe Propuesta 2 – TPO Programación III

1. Funcionalidad 1: organización y consulta de pedidos

1.1 Análisis del problema

Cada pedido tiene un identificador, un centro de destino, una fecha límite, una prioridad y un peso total. El sistema debe ordenar los pedidos por fecha límite más cercana, ordenarlos por prioridad descendente y buscar un pedido por su identificador.

La búsqueda por identificador tiene que ser rápida aunque haya muchos pedidos, y para eso se usa búsqueda binaria, que solo funciona sobre datos ordenados. Por eso la lista de pedidos se mantiene ordenada por id en todo momento, insertando cada pedido nuevo en su posición en lugar de agregarlo al final. El costo temporal de agregar es de O(N) (siguiendo la estrategia de correr los elementos para hacer lugar al nuevo) y en cada búsqueda O(log N). Esta solución es superior a la opción de agregar al final del arreglo y ordenar antes de cada búsqueda O(N log N) por búsqueda o buscar recorriendo toda la lista O(N) por búsqueda; en ambas cada consulta por id sale más cara.

Reglas de desempate:

- **Por prioridad** (definida por la consigna): ante igual prioridad gana la fecha límite más cercana y, si también coincide, el identificador menor.
- **Por fecha** (por consecuencia del criterio anterior): ante igual fecha gana el identificador menor.

OBS: Los identificadores son únicos. Se lanza una excepción si se quiere insertar un pedido con un id ya existente.

1.2 Organización de los datos que se necesita en cada función principal

- **Buscar por id:** necesita una lista ordenada por id, porque la búsqueda binaria solo funciona con datos ordenados. GestorPedidos mantiene una única lista, pedidosPorId, siempre ordenada por id. Costo: O(log N).
- **Ordenar por fecha o por prioridad:** no necesita ninguna organización previa, porque recibe los datos y los ordena. Se ordena una copia de la lista y la lista interna no se modifica. Costo: O(N log N).
- **Agregar un pedido:** tiene que conservar el orden por id y detectar ids repetidos. Se busca la posición donde va el pedido y se lo inserta ahí, sin reordenar toda la lista. Costo: O(N).

La lista interna debe quedar siempre ordenada por id. Por eso los ordenamientos por fecha y por prioridad se calculan sobre una copia: si se ordenara la lista interna, dejaría de estar ordenada por id y la búsqueda binaria dejaría de funcionar.

1.3 Estrategia elegida: Divide y Conquista

Ordenamiento: Merge Sort. Divide la lista en dos mitades, ordena cada mitad recursivamente y combina las dos mitades ya ordenadas intercalándolas con merge. El caso

base es una sublista de un solo elemento. Se eligió porque garantiza O(N log N) sin importar el orden en que lleguen los datos.

Búsqueda: búsqueda binaria. Compara el id buscado con el del elemento del medio de la lista ordenada y descarta la mitad donde no puede estar; repite sobre la mitad que queda. El caso base es un solo elemento. Reduce el problema a la mitad en cada paso.

La estrategia se reconoce en el código: MergeSort.mergeSort hace dos llamadas recursivas sobre las mitades y luego llama a merge, y BusquedaBinaria.buscar se llama a sí misma sobre una de las mitades. ComparadorPedido comprende las reglas de orden (fecha, prioridad e id), de modo que el mismo Merge Sort sirve para ambos ordenamientos.

1.4 Alternativas descartadas

- **Selección e inserción:** son $O(N^2)$ y se vuelven impracticables al crecer la cantidad de pedidos.
- **Quick Sort:** es O(N log N) en promedio pero $O(N^2)$ en el peor caso.
- **Búsqueda lineal:** es O(N) y no aprovecha que la lista está ordenada por id.
- **Reordenar toda la lista en cada alta:** costaría O(N log N) por pedido agregado, contra O(N) de insertarlo en su posición.

1.5 Complejidad temporal

N es la cantidad de pedidos. Merge Sort y la búsqueda binaria son algoritmos recursivos de Divide y Conquista, así que su tiempo se obtiene con el teorema maestro, que usa tres valores:

- **a:** la cantidad de subproblemas en que se divide el problema.
- **b:** el factor en que se reduce el tamaño en cada subproblema (cada uno tiene tamaño N/b).
- **k:** el grado del polinomio que describe el costo del trabajo que se hace fuera de las llamadas recursivas, que es $O(N^k)$.

El resultado depende de cómo se compara a con b^k:

$$T(N) = \begin{cases} O(N^k) & \text{si } a < b^k \\ O(N^k \log N) & \text{si } a = b^k \\ O\left(N^{\log_b a}\right) & \text{si } a > b^k \end{cases}$$

Merge Sort. Divide la lista en 2 mitades (a = 2) de tamaño N/2 (b = 2). El trabajo fuera de las llamadas recursivas es el de merge, que recorre las dos mitades una vez y cuesta O(N); calcular el punto medio es O(1). Entonces ese trabajo es O(N) (k = 1).

Como $b^k = 2^1 = 2$ es igual a a, se aplica el caso $a = b^k$:

$$a = 2, \quad b^k = 2^1 = 2 \;\Rightarrow\; a = b^k \;\Rightarrow\; T(N) = O(N^k \log N) = O(N \log N)$$

Búsqueda binaria. Se divide en 2 mitades pero solo se llama a una (a = 1) de tamaño N/2 (b = 2), y fuera de la llamada recursiva solo compara con el elemento del medio, un trabajo constante, $O(1) = O(N^0)$, así que k = 0.

Como $b^k = 2^0 = 1$ es igual a a, se aplica otra vez el caso $a = b^k$:

$$a = 1, \quad b^k = 2^0 = 1 \Rightarrow a = b^k \Rightarrow T(N) = O(N^k \log N) = O(N^0 \log N) = O(\log N)$$

- **Ordenar por fecha o por prioridad:** copiar la lista es O(N) y ordenarla O(N log N), en total O(N log N).
- **Agregar un pedido:** hallar la posición recorre hasta N elementos y insertar desplaza hasta N, en total O(N). Cargar N pedidos de a uno es $O(N^2)$ en el peor caso. Se acepta porque la consulta por id, que es la operación que se repite, es O(log N).

1.6 Casos de prueba

PruebasPedidos ejecuta 44 verificaciones en 15 grupos:

- **Empates:** si dos pedidos tienen la misma fecha o la misma prioridad, el orden lo decide el desempate; y si todo es igual, va primero el id menor.
- **Datos ya ordenados o al revés:** los ordenamientos funcionan aunque los pedidos ya vengan en orden o en orden inverso.
- **Buscar por id:** se encuentran el primero, el del medio y el último. Un id que no existe (uno que cae entre dos existentes, menor que el mínimo, mayor que el máximo o negativo) devuelve que no hay pedido.
- **Lista vacía o con un solo pedido:** ordenar y buscar funcionan igual.
- **Id repetido:** al agregar un pedido con un id que ya existe se lanza una excepción y la lista queda como estaba.
- **Datos originales:** ordenar no modifica el arreglo de entrada ni la lista del gestor; el resultado es una copia.
- **Agregar después de buscar:** al insertar un pedido nuevo, la lista sigue ordenada por id.
- **Un orden no altera el otro:** pedir el orden por fecha no cambia el orden por prioridad, y al revés.
- **Comparador:** un criterio inválido lanza excepción, y los desempates se prueban comparando dos pedidos directamente.
- **Volumen (2000 pedidos):** con ids mezclados y muchas fechas y prioridades repetidas, los dos órdenes quedan correctos y se encuentra cada id.

2. Funcionalidad 2: cálculo de la ruta más rápida

2.1 Análisis del problema

La organización dispone de una red vial formada por depósitos, centros comunitarios y conexiones dirigidas. Cada conexión habilitada posee un tiempo de viaje fijo y positivo, expresado en minutos. Dados un origen y un destino, se busca el recorrido cuya suma de tiempos sea mínima, no necesariamente el que tenga menos conexiones.
El sistema debe mostrar las ubicaciones atravesadas y el tiempo total. También debe permitir deshabilitar una conexión sin modificar automáticamente el sentido inverso, recalcular con la red actualizada y avisar cuando el destino no sea alcanzable. Los identificadores de las ubicaciones son únicos, y no se admiten tiempos cero o negativos para conexiones habilitadas.
La red se representa con una matriz de adyacencia: tiempos [i] [j] almacena los minutos para ir de la ubicación i a la j y 0 significa que no hay conexión habilitada. Los identificadores externos se guardan en una lista de ubicaciones, que permite relacionarlos con las posiciones de la matriz. La matriz refleja la dirección de circulación: tiempos [i] [j] y tiempos[j] [i] son independientes.

2.2 Estrategia elegida: Greedy (algoritmo de Dijkstra)

Se utilizó la estrategia Greedy con el algoritmo de Dijkstra: en cada iteración elige entre las ubicaciones no visitadas aquella con menor tiempo acumulado conocido desde el origen, y confirma ese tiempo como definitivo. Esto es correcto porque todos los tiempos de las conexiones son positivos.

- **Candidatos:** las ubicaciones todavía no visitadas que tienen un tiempo conocido desde el origen.
- **Selección:** escoger la ubicación pendiente con el menor tiempo acumulado (en un empate, se elige el identificador menor para obtener un resultado reproducible).
- **Factibilidad y actualización:** revisar las conexiones habilitadas que salen de la ubicación elegida. Si pasar por ella mejora el tiempo de una ubicación vecina, se actualizan su distancia y su predecesor.
- **Solución:** terminar cuando el destino queda confirmado, o cuando no existen más ubicaciones alcanzables. Se reconstruye el recorrido desde el destino siguiendo los predecesores.

Se mantienen tres arreglos principales: distancia [] (tiempo mínimo conocido, inicialmente infinito salvo origen = 0), visitado [] (ubicaciones definitivas) y anterior [] (predecesor que permite reconstruir el camino). La selección y la actualización se realizan con ciclos sobre la matriz; no se utilizan métodos ajenos a las diapositivas ni bibliotecas externas de grafos.
Ejemplo de ejecución. Se consideran las conexiones dirigidas 1→2 (6), 1→3 (16), 1→4 (30), 2→3 (4), 2→4 (14) y 3→4 (3), con origen 1 y destino 4. La ubicación 5 no posee conexiones.

Paso	Se confirma	Tiempo fijo	Mejoras observadas
1	1	0	2=6; 3=16; 4=30
2	2	6	3: 16→10; 4: 30→20
3	3	10	4: 20→13
4	4	13	Destino confirmado; fin

El recorrido óptimo es 1→2→3→4 y tarda 6 + 4 + 3 = 13 minutos. El algoritmo registra en texto cada elección confirmada y cada mejora, de modo que se pueden visualizar las decisiones principales, como pide la consigna.
Si se deshabilita la conexión 2→3, se vuelve a ejecutar Dijkstra sobre la matriz modificada. El nuevo recorrido es 1→3→4 y tarda 16 + 3 = 19 minutos. Si se consulta la ubicación aislada 5 desde 1, se informa que no existe una ruta disponible.

2.3 Alternativas descartadas

- **Elegir siempre la conexión saliente más rápida:** es una decisión local que no garantiza el mejor tiempo completo hasta el destino; puede llevar a un camino globalmente más lento..
- **Floyd:** también fue visto en la materia y calcula caminos mínimos entre todos los pares. Sería apropiado para muchas consultas sobre la misma red (la funcionalidad opcional A1), pero aquí se consulta un origen y destino y pueden modificarse conexiones, por lo que no hace falta calcular toda la matriz de pares.
- **Enumerar todas las rutas:** examinar las combinaciones posibles resulta mucho más costoso en redes crecientes que el procedimiento Greedy de Dijkstra.

2.4 Complejidad temporal

Sea V la cantidad de ubicaciones. Con la matriz de adyacencia, en cada una de hasta V iteraciones se recorren V posiciones para seleccionar la próxima ubicación y hasta V posiciones para revisar sus conexiones. Por lo tanto, el costo temporal del cálculo es $O(V^2)$.Reconstruir el camino cuesta O(V) en el peor caso: primero se guardan las ubicaciones en orden inverso siguiendo los predecesores y después se recorren de atrás hacia adelante para formar la ruta final, sin inserciones al inicio de la lista. Deshabilitar una conexión existente cuesta O(1), ya que solo se modifica una celda de la matriz. Luego, recalcular la ruta vuelve a costar $O(V^2)$. Agregar una ubicación implica ampliar y copiar la matriz, con un costo de $O(V^2)$.

2.5 Decisiones de diseño

- **Dirección de las conexiones:** una conexión 1→2 no implica que exista 2→1. Deshabilitar una solo modifica la celda correspondiente.
- **Valores especiales:** el 0 de la matriz significa ausencia de conexión; las distancias todavía desconocidas usan infinito. Para sumar tiempos se emplea long y evitar el desbordamiento de int.
- **Empates:** entre dos ubicaciones pendientes con la misma distancia se confirma primero la de identificador menor. Si un recorrido alternativo iguala un tiempo ya encontrado, se conserva el predecesor previo; ambos recorridos son óptimos.
- **Casos límite y validaciones:** si el origen coincide con el destino, el recorrido contiene solo esa ubicación y tarda 0. Se rechazan la red nula, las ubicaciones inexistentes, las conexiones con tiempos no positivos y los identificadores repetidos.

- **Organización del código:** RedVial administra los datos de la red, CalculadorRutas implementa Dijkstra y ResultadoRuta almacena los resultados. Para mejorar la legibilidad, el método calcular delega tareas en tres métodos auxiliares: elegirUbicacion, actualizarDistancias y reconstruirRecorrido. Main permite demostrar el funcionamiento y PruebasRutas verifica los resultados.

2.6 Casos de prueba

- **Ejemplo de demostración:** se calcula el recorrido 1→2→3→4 de 13 minutos.
- **Recalcular después de deshabilitar:** sin 2→3 se obtiene 1→3→4 en 19 minutos; con otra modificación se obtiene 1→2→4 en 20 minutos; al rehabilitar vuelve el óptimo de 13.
- **Sentido de circulación:** una conexión de ida no habilita automáticamente la vuelta, y deshabilitar una dirección no altera la opuesta.
- **Sin ruta y origen igual a destino:** se informa que una ubicación aislada es inalcanzable, y que viajar desde una ubicación hasta ella misma tarda 0.
- **Otros caminos y empates:** se comprueban caminos directos, rutas alternativas y un caso con soluciones de igual duración.
- **Decisiones y datos:** se revisa que la traza contenga confirmaciones y mejoras, y que las listas devueltas no permitan alterar el resultado interno.
- **Validaciones y volumen:** se comprueban entradas inválidas, tiempos grandes y una red de 200 ubicaciones cuyo recorrido más rápido tiene 199 minutos.

3. Funcionalidad 3: selección de cargas prioritarias

3.1 Análisis del problema

Un vehículo tiene una capacidad máxima de carga (un entero mayor o igual a 0). Cada lote tiene un identificador, un peso entero positivo y una prioridad entera no negativa. Los lotes no se pueden fraccionar y cada uno se carga una sola vez. El objetivo es elegir los lotes que den la mayor prioridad total sin superar la capacidad, y mostrar la selección, el peso total, la prioridad obtenida y la capacidad restante. Es el problema de la mochila con resolución mediante programación dinámica

El sistema rechaza con una excepción los datos que contradicen estas condiciones: peso no positivo, prioridad negativa, capacidad negativa, vehículo o lista nulos, lotes nulos e ids de lote repetidos.

OBS: Si hay varias selecciones con la misma prioridad total, se elige la que usa los lotes que aparecen primero en la lista. Por ejemplo, con capacidad 5 y los lotes L1 (5, 10), L2 (3, 5) y L3 (2, 5), la lista en ese orden elige [L1] y la lista invertida elige [L3, L2]; la prioridad es 10 en ambos casos. Un lote de prioridad 0 nunca se carga, porque no aporta prioridad.

3.2 Organización de los datos

La selección usa tres estructuras:

- **Lista de lotes:** se accede por posición, porque la fila i de la matriz corresponde al lote i−1 de la lista. La lista original no se modifica.

- **Matriz de enteros matriz[i][c]:** tiene n+1 filas (una por lote, más la fila 0 de "ningún lote") y C+1 columnas (una por capacidad, de 0 a C). Guarda la mayor prioridad posible para cada par (lotes disponibles, capacidad).
- **Arreglo de booleanos:** al reconstruir la solución marca qué lotes se cargaron, y con él se arma la lista final respetando el orden original de los lotes.

3.3 Estrategia elegida: Programación Dinámica

La idea es armar una tabla, matriz[i][c], donde el valor de cada celda representa la mayor prioridad si solo puedo usar los primeros i lotes y tengo una capacidad c. La fila 0 (ningún lote) vale 0 en todas las columnas, y la fila i corresponde al lote i−1 de la lista. La tabla se llena fila por fila, de menor a mayor y cada celda se calcula teniendo en cuenta las celdas de la fila anterior, que ya están resueltas.

Para calcular una celda hay solo dos opciones con el lote i:

- **No cargarlo:** queda lo mismo que sin este lote, es decir el valor de la celda de arriba, matriz[i-1][c].
- **Cargarlo:** se suma la prioridad del lote. El resultado es su prioridad más matriz[i-1][c - peso]. Solo es posible si el lote entra, es decir si la suma de los pesos es menor o igual a c.

Si el lote entra, se elige el máximo de las dos opciones. Si el lote no entra, la única opción es no cargarlo.

Con w_i el peso y v_i la prioridad del lote i, esto se define así:

$$M[i][c] = \begin{cases} M[i-1][c] & \text{si } w_i > c \\ \max(M[i-1][c],\ M[i-1][c-w_i] + v_i) & \text{si } w_i \leq c \end{cases}$$

La prioridad máxima es la última celda, matriz[n][C]. Para saber qué lotes se cargaron se recorre la matriz desde esa celda hacia arriba: si matriz[i][c] es distinto de la celda de arriba (matriz[i-1][c]), el lote i se cargó y la capacidad baja en su peso; si es igual, no se cargó. Por eso, cuando dos selecciones tienen la misma prioridad, gana la que usa los lotes que aparecen primero en la lista.

Ejemplo. Lotes (peso, prioridad): L1 (1, 10), L2 (4, 3), L3 (5, 5), L4 (7, 8), con capacidad 8.

i \ c	0	1	2	3	4	5	6	7	8
0 (ninguno)	0	0	0	0	0	0	0	0	0
1 (L1)	0	10	10	10	10	10	10	10	10
2 (L2)	0	10	10	10	10	13	13	13	13
3 (L3)	0	10	10	10	10	13	15	15	15
4 (L4)	0	10	10	10	10	13	15	15	18

La prioridad máxima es 18. Reconstrucción: en (4, 8) el valor 18 es distinto del 15 de arriba, así que se carga L4 como parte de la solución y la capacidad pasa a 8 − 7 = 1. Se mueve a la celda (3, 1) y se compara con (2, 1) donde el valor 10 es igual al de arriba, así que L3 y L2 no se cargan. En la celda (1, 1) el valor 10 es distinto del 0 de arriba, así que se carga L1 y la capacidad queda en 0. Como solución quedan L1 y L4, peso 8, prioridad 18, capacidad restante 0.

Por qué Programación Dinámica. Para decidir qué hacer con un lote alcanza con conocer la mejor carga con los lotes anteriores y la capacidad que queda, que es el mismo problema pero más chico. Esos problemas chicos se repiten: en el ejemplo, la celda (2, 1) se usa para calcular tanto la celda (3, 1) como la (3, 6). Por eso cada uno se resuelve una sola vez y se guarda en la tabla, y se reutiliza cada vez que hace falta.

La estrategia se reconoce en SelectorCargas: llenarMatriz calcula la matriz con el caso base de la fila 0 y la elección entre cargar y no cargar con Math.max, y reconstruir recorre la matriz hacia atrás para encontrar la solución.

3.4 Alternativas descartadas

La otra alternativa considerada fue una estrategia Greedy, en la que se ordenan los lotes por la relación prioridad/peso (de mayor a menor) y se cargan en ese orden mientras entren en la capacidad del vehículo.

Se descartó porque no garantiza la solución óptima. Greedy decide mirar un lote a la vez y nunca se arrepiente: una vez que carga un lote, no puede revisar si convenía más dejar ese espacio para otros. Como cada lote se carga entero, el lote con mejor relación en ese momento puede ocupar un espacio que luego deja capacidad sin aprovechar.

Ejemplo: capacidad 10 y los lotes A (peso 6, prioridad 60), B (peso 5, prioridad 45) y C (peso 5, prioridad 45). Las relaciones prioridad/peso son 10, 9 y 9, así que Greedy elige primero A. Después de A quedan 4 de capacidad y ni B ni C entran (pesan 5), por lo que el resultado es prioridad 60 con 4 unidades sin usar. Pero la mejor selección era B y C ya que pesan 5 + 5 = 10 y suman prioridad 90.

La Programación Dinámica no tiene este problema porque, para cada lote y cada capacidad, calcula las dos posibilidades (cargarlo o no cargarlo) y se queda con la mejor, en lugar de comprometerse con una sola decisión.

3.5 Complejidad temporal

Siendo n la cantidad de lotes y C la capacidad del vehículo.

- **Llenar la matriz:** llenarMatriz tiene dos for anidados. El de afuera recorre los lotes (i de 1 a n), es decir n vueltas. El de adentro recorre las capacidades (c de 0 a C), es decir C+1 vueltas por cada vuelta del de afuera. Dentro del for interno solo hay una comparación, una suma y un Math.max, que son operaciones constantes que cuestan O(1). Entonces el total es Θ(n·C).
- **Reconstrucción y armado del resultado:** se recorre la matriz una fila por vez, de la última a la primera, y en cada fila se hace una comparación y, a lo sumo, una resta. Son n pasos de tiempo constante, es decir Θ(n). Armar la lista de lotes

elegidos recorre los n lotes una vez, también Θ(n). Complejidad final: Θ(n) + O(n) = Θ(n)

- **Validaciones:** recorren los lotes una vez, comprobando que ninguno sea nulo y que su id no haya aparecido antes (con un HashSet,cada consulta es O(1)). La complejidad final es O(n).
- **Seleccionar :** se suman los tres costos, Θ(n·C) + Θ(n) + O(n). Así que el total es Θ(n·C).

3.6 Casos de prueba

PruebasCargas ejecuta 52 verificaciones en 14 grupos:

- **Ejemplo en clase:** con capacidad 8 y los lotes del ejemplo se eligen los lotes 1 y 4, con prioridad 18, peso 8 y capacidad restante 0.
- **Caso donde Greedy no sirve y programación dinámica si:** con capacidad 10 se eligen los lotes 2 y 3, con prioridad 90.
- **Sobra capacidad:** con los mismos lotes y capacidad 9, se elige solo el lote 1 (prioridad 60) y quedan 3 libres.
- **Casos límite:** capacidad 0, lista vacía, un lote más pesado que la capacidad, ningún lote que entre, un lote que entra justo y todos los lotes entrando.
- **Prioridad 0:** un lote sin prioridad nunca se carga.
- **Empates:** si dos selecciones suman la misma prioridad, gana la que usa los lotes que aparecen primero; con la lista invertida cambia la selección pero no la prioridad.
- **Datos originales:** la lista de lotes no se modifica.
- **Datos inválidos:** peso 0 o negativo, prioridad negativa, capacidad negativa, vehículo nulo, lista nula, lote nulo e id repetido lanzan excepción; la capacidad 0 es válida.
- **Volumen (2000 lotes, capacidad 500):** todos pesan 1 y las prioridades 1 a 2000 se reparten al azar; se eligen 500 lotes, todos entre los de prioridad 1501 a 2000, con prioridad total 875250 y capacidad restante 0.